

SIMATS ENGINEERING

CO 1 Assessment Tool 3

Game Based Learning Assessment

Course: Artificial Intelligence and Expert Systems

Course Code: MLA0103

NAME : YAMEENULLAH M H

REG NO : 192525350

Table of Contents

1. Introduction & How to Use This Guide
2. Foundations of AI Search — Core Concepts
3. Game 1 — AI Maze Escape
4. Game 2 — 12-Queens Challenge
5. Game 3 — Water Jug Puzzle
6. Game 4 — Connect Four AI Challenge
7. Game 5 — 8-Puzzle AI Challenge
8. Cross-Game Comparison Table
9. Glossary of Key AI Terms
10. How to Write Up Each Section of Your Report
- Appendix A: Official Evaluation Rubric (Reproduced)
- Appendix B: Rubric Self-Check Before Submission
- Appendix C: Code of Conduct & Marks Sheet (Reproduced)
13. References & Further Reading

1. Introduction & How to Use This Guide

This guide is a companion to the SIMATS Assessment Tool 3 (Game Based Learning Assessment) for the course Artificial Intelligence and Expert Systems. The official assessment gives you five short scenarios and asks you to play a game, record your performance, and analyse it using AI search concepts. This guide does not replace that report — instead, it gives you everything you need to understand each game deeply before you play it: the underlying AI concept, a worked example, a diagram of the idea, tips for playing efficiently, and reflection questions that map directly onto the report's rubric criteria.

Each of the five chapters that follow (Chapters 3–7) is structured the same way, so you can move between games without losing your place:

- Platform & Direct Link — click straight through to the game
- AI Concept / Domain — the formal search category the puzzle belongs to
- Diagram — a visual model of the problem
- Scenario — an expanded description of what you are actually doing
- Objective, Initial State & Goal State, Actions/Operators — the formal problem definition
- Algorithm Walkthrough — how a computer would actually solve this, step by step, with pseudocode
- Worked Example Trace — a sample sequence of states/moves toward the goal
- How Playing This Helps You Learn AI — the transferable skill
- Common Mistakes — pitfalls students typically hit
- Real-World Applications — where this search idea shows up outside the game
- Tips to Play & Solve Efficiently
- Reflection Questions for Your Report — ready-made prompts for the rubric's analysis section

Chapter 2 first lays a shared foundation — the vocabulary and algorithms (state space, BFS, DFS, A*, CSP, Minimax) that all five games draw from — so the per-game chapters can focus on how each concept specifically applies to that puzzle.

A note on academic integrity: the 'Step-by-Step Solution', 'Game Performance', and 'Successful Completion and Evidence' sections of the official report must reflect your own actual play-through of each game, including your own screenshots and move counts. This guide's Worked Example Traces are illustrative models showing the level of detail expected — not a substitute for playing the games and recording your own results. Use the Fillable Score Sheet at the end of each chapter to capture your genuine performance data as you play.

2. Foundations of AI Search — Core Concepts

Almost every classical AI problem-solving technique can be described using the same small vocabulary: a state space made up of states, connected by actions (also called operators), starting from an initial state and aiming at one or more goal states. The five games in this assessment are simply five different 'costumes' worn by this same underlying idea.

2.1 State Space, States, and Operators

A state is a complete snapshot of the problem at one moment — the robot's current cell in a maze, the arrangement of queens on a board, the water levels in two jugs, the positions of discs on a Connect Four grid, or the arrangement of tiles in the 8-Puzzle. An operator (or action) is a rule that transforms one state into another — moving one step, placing one queen, pouring one jug into another, dropping one disc, sliding one tile. The state space is the graph formed by every reachable state connected by every legal operator.

Generic State-Space Graph: Nodes = States, Edges = Actions

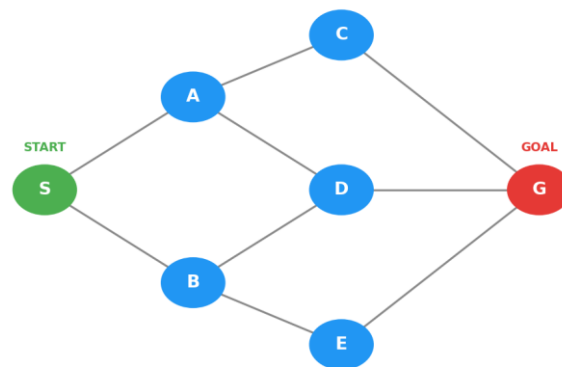


Fig. 2.1 — A generic state-space graph. Solving a problem means finding a path from the *START* node to a *GOAL* node.

Searching for a solution means exploring this graph — expanding states one by one — until a goal state is found. The two big questions that separate one search algorithm from another are: (1) in what order do we expand states? and (2) do we use any extra information to decide which state looks most promising?

2.2 Uninformed Search: BFS and DFS

Uninformed (or 'blind') search algorithms explore the state space using no information beyond the problem definition itself — they don't know which direction is 'closer' to the goal, only which states are reachable next.

Breadth-First Search (BFS) expands all states at the current depth before moving to the next depth level, using a First-In-First-Out (FIFO) queue. Because it explores level by level, BFS is guaranteed to find the shortest path (in terms of number of steps) in an unweighted graph — this is exactly why it is the natural fit for the AI Maze Escape and Water Jug puzzles in this assessment.

```
function BFS(start, goal):  
    queue <- [start]  
    visited <- {start}
```

```

while queue is not empty:
    node <- queue.dequeue()
    if node == goal: return path_to(node)
    for each neighbor of node:
        if neighbor not in visited:
            visited.add(neighbor)
            queue.enqueue(neighbor)
return failure

```

Depth-First Search (DFS) instead dives as deep as possible along one branch before backtracking, using a Last-In-First-Out (LIFO) stack (or recursion). DFS uses less memory than BFS but does not guarantee the shortest path — it may find a much longer route to the goal simply because it committed early to one direction.

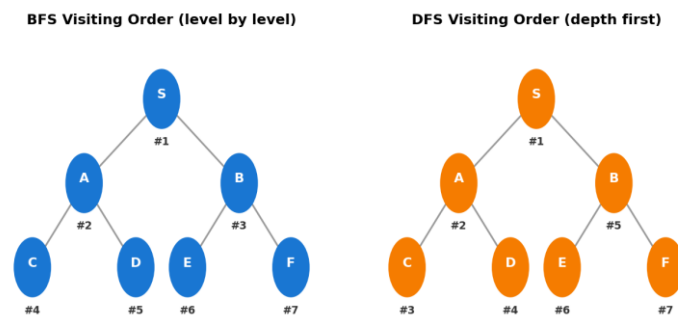


Fig. 2.2 — The same tree explored by BFS (left, level-by-level) versus DFS (right, depth-first). Numbers show visiting order.

2.3 Informed (Heuristic) Search: A*

Informed search algorithms use a heuristic function $h(n)$ — an estimate of how far a state n is from the goal — to prioritise which state to expand next. The A* algorithm combines the cost already paid to reach a state, $g(n)$, with the estimated remaining cost, $h(n)$, and always expands the state with the lowest total $f(n) = g(n) + h(n)$. A* is the classical approach to the 8-Puzzle, where $h(n)$ is typically the count of misplaced tiles or the sum of Manhattan distances of every tile from its goal position.

```

function A_STAR(start, goal):
    open <- priority_queue([start], key=f)
    g[start] <- 0
    while open is not empty:
        node <- open.pop_lowest_f()
        if node == goal: return path_to(node)
        for each neighbor of node:
            tentative_g <- g[node] + cost(node, neighbor)
            if tentative_g < g.get(neighbor, infinity):
                g[neighbor] <- tentative_g
                f[neighbor] <- tentative_g + h(neighbor)
                open.push(neighbor)
    return failure

```

A heuristic is called admissible if it never overestimates the true remaining cost — this guarantees A* still finds the optimal (shortest) solution while typically exploring far fewer states than blind search.

2.4 Constraint Satisfaction Problems

A Constraint Satisfaction Problem (CSP) is defined by a set of variables, a domain of possible values for each variable, and a set of constraints that restrict which combinations of values are allowed. The N-Queens problem is a textbook CSP: the variables are the queens (or, equivalently, the rows), the domain for each is 'which column', and the constraints forbid any two queens sharing a row, column, or diagonal. CSPs are typically solved with backtracking search — a depth-first exploration of partial assignments that abandons (backtracks from) any assignment that violates a constraint as early as possible, instead of waiting until the whole board is filled.

Backtracking Search Tree (Conceptual, 4-Queens)

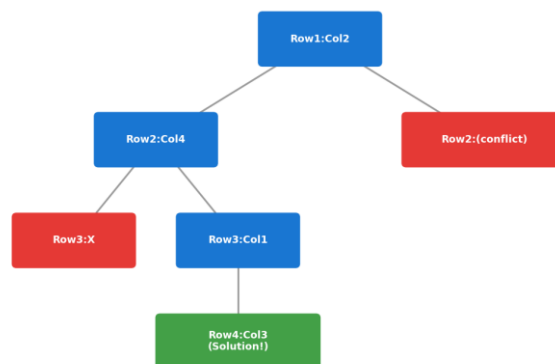


Fig. 2.3 — Backtracking abandons a branch as soon as a constraint is violated (red), and continues exploring until a full valid assignment is found (green).

```
function BACKTRACK(assignment):
    if assignment is complete: return assignment
    var <- select_unassigned_variable()
    for each value in domain(var):
        if value is consistent with assignment (no constraint
violated):
            add {var = value} to assignment
            result <- BACKTRACK(assignment)
            if result != failure: return result
            remove {var = value} from assignment // backtrack
    return failure
```

Backtracking can be made dramatically faster with techniques such as constraint propagation (removing values from a variable's domain as soon as they become impossible) and variable ordering heuristics (e.g., always assign the most constrained row next).

2.5 Adversarial Search: Minimax & Alpha-Beta Pruning

When a problem involves an opponent actively trying to prevent you from reaching your goal — as in Connect Four — the search must account for the opponent's best possible response to every move. The Minimax algorithm assumes the current player (MAX) tries to maximise the

final outcome, while the opponent (MIN) tries to minimise it. The algorithm explores the game tree to some depth, evaluates the resulting positions, and propagates values back up: MIN nodes take the minimum of their children, MAX nodes take the maximum.

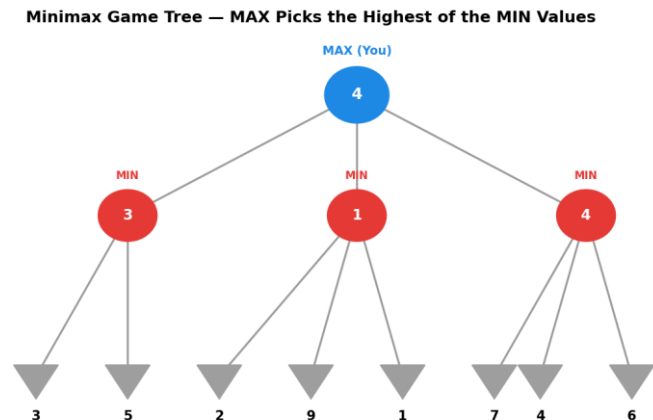


Fig. 2.4 — A minimax tree. MIN nodes choose the smallest child value; the MAX root then chooses the largest of those.

```
function MINIMAX(node, depth, maximizing):
    if depth == 0 or node is terminal:
        return evaluate(node)
    if maximizing:
        best <- -infinity
        for each child of node:
            best <- max(best, MINIMAX(child, depth-1, false))
        return best
    else:
        best <- +infinity
        for each child of node:
            best <- min(best, MINIMAX(child, depth-1, true))
        return best
```

Because a full game tree is usually far too large to search completely, Alpha-Beta Pruning is used to skip branches that cannot possibly influence the final decision, letting the algorithm search much deeper within the same time budget without changing the result.

2.6 Comparing the Algorithms at a Glance

Before diving into the five games individually, it helps to see all the foundational algorithms side by side. Each one answers the same underlying question — 'which state should I expand next?' — in a different way.

Algorithm	Uses a Heuristic?	Handles an Opponent?	Typical Use in This Assessment
BFS	No	No	Maze Escape, Water Jug Puzzle

Algorithm	Uses a Heuristic?	Handles an Opponent?	Typical Use in This Assessment
DFS	No	No	Maze Escape (alternative), Backtracking base
Backtracking	No (uses constraints)	No	12-Queens Challenge
A*	Yes	No	8-Puzzle
Minimax / Alpha-Beta	Optional (evaluation function)	Yes	Connect Four

Two properties are worth remembering for every algorithm you study: completeness (will it always find a solution if one exists?) and optimality (if it finds a solution, is it guaranteed to be the best one?). BFS, backtracking, and A* (with an admissible heuristic) are both complete and optimal for the problems in this assessment; DFS is complete but not optimal; Minimax is optimal only within the depth it actually searches.

2.7 A Note on Problem-Solving Strategy

Across all five games, the same three-step discipline helps regardless of which algorithm you are conceptually applying:

1. Define the problem formally first — write down the initial state, the goal state, and the available actions before making a single move.
2. Track what you've already tried — whether that's visited maze cells, tested queen placements, or jug states, so you never repeat wasted effort.
3. Compare your result to the theoretical optimum — after solving, ask 'could an algorithm have done this in fewer steps, and why?' This is exactly what the report's 'Efficiency and Optimality' and 'Critical Thinking' rubric criteria are testing.

- Fig. 3.1 — The robot must travel from START (green) to GOAL (red) avoiding walls.

- Move Right — if the cell to the right is open

Algorithm Walkthrough — Breadth-First Search (BFS)

1. Place the START cell in a queue and mark it visited.
2. Remove the cell at the front of the queue.
3. If it is the GOAL cell, stop — the path used to reach it is the shortest path.
4. Otherwise, look at its four neighbours (up/down/left/right).
5. For every neighbour that is open and not yet visited, mark it visited, record which cell led to it, and add it to the back of the queue.
6. Repeat from step 2 until the goal is found or the queue is empty (no path exists).

Worked Example Trace

Suppose START is at grid cell (0,0) and GOAL is at (2,6). BFS first explores every cell exactly one step away from (0,0): (0,1) and (1,0), if open. It then explores every cell two steps away, and so on, expanding outward in 'rings' around the start.

The moment the GOAL cell is first reached in this ring-by-ring expansion, the number of rings crossed equals the length of the shortest path — for example, if GOAL first appears in ring 9, then 9 is the minimum possible number of moves, and tracing back through the recorded predecessor of each cell reconstructs the actual shortest route.

Step	Current Cell	Action Taken	Resulting Cell
1	(0,0)	Move Down	(1,0)
2	(1,0)	Move Down	(2,0)
3	(2,0)	Move Right	(2,1)
4	(2,1)	Move Right	(2,2)
...	...	...	...
n	(current)	Move (final)	GOAL

Complexity & Performance Notes

Let b be the branching factor (up to 4 possible moves per cell) and d be the depth of the shortest solution. BFS explores states level by level, so in the worst case it visits every cell at depth $\leq d$ before finding the goal.

Approach	Time Complexity	Space Complexity	Optimal?
BFS	$O(b^d)$	$O(b^d)$	Yes — shortest path guaranteed
DFS	$O(b^m)$, $m = \text{max maze depth}$	$O(bm)$	No — may find a longer path
Random walk (no memory)	Unbounded / not guaranteed to terminate	$O(1)$	No

Alternative Approaches You Could Try

- Dijkstra's Algorithm — behaves like BFS here since every move has equal cost, but would matter if some cells were 'harder' to cross (e.g., mud vs. open floor) with different move costs.
- A* Search — if you can estimate straight-line distance to the goal, A* reaches the goal faster than BFS by exploring fewer irrelevant cells, while still guaranteeing the shortest path.
- Bidirectional Search — simultaneously searching forward from START and backward from GOAL can meet in the middle, often exploring far fewer total cells than one-directional BFS.

How Playing This Helps You Learn AI

- Makes the abstract idea of a 'state space' concrete: every maze cell is a state, every legal move is an edge.
- Demonstrates why BFS guarantees the shortest path in an unweighted grid, while DFS may find a valid but longer path faster.
- Builds intuition for frontier expansion — you can visually track which cells an algorithm would explore next.
- Encourages comparing your manual solution against an optimal algorithmic solution, reinforcing the concept of solution optimality.

Common Mistakes to Avoid

- Re-entering a cell you have already visited, wasting moves and possibly getting stuck in a loop.
- Assuming the first path you find is automatically the shortest — only a systematic method like BFS guarantees this.
- Forgetting to record the total step count needed for the report's efficiency analysis.

Real-World Applications of This Technique

- GPS and map navigation apps computing the shortest driving or walking route.
- Network routing protocols finding the fewest-hop path between two computers.
- Robotics path planning in warehouses and autonomous vehicles.

Frequently Asked Questions

Q: Does DFS ever find the shortest path?

A: Only by coincidence. DFS commits to one direction and may reach the goal via a much longer route before backtracking would have revealed a shorter one.

Q: What if there are multiple shortest paths?

A: BFS will find one of them (the specific one depends on the order neighbours are checked), but all shortest paths have the same length.

Q: Why is a queue used for BFS instead of a stack?

A: A queue processes states in the order they were discovered (FIFO), which is what guarantees level-by-level expansion. A stack (LIFO) is what turns the same algorithm into DFS.

Tips to Play & Solve Efficiently

- Before moving, mentally mark dead ends so you don't re-enter them.

- Sketch the maze on paper and simulate BFS ring-by-ring to find the true shortest path, then compare with your in-game attempt.
- Count your total moves after finishing — this becomes your 'path cost' for the efficiency section.

Reflection Questions for Your Report

1. How many moves did your path take, and how does it compare with the shortest possible path you can identify on paper?
2. At which points did you have to backtrack, and what does that suggest about how a DFS-based robot might have behaved differently?
3. If the maze were twice as large, how would the number of states BFS must explore change?

Fillable Score Sheet — Record Your Own Attempt

Use this table while you play to capture the data you will need for the 'Game Performance' and 'Efficiency and Optimality' sections of your report.

Field	Your Recorded Value
Total moves taken	
Time taken (mm:ss)	
Number of dead ends hit	
Shortest possible path length (your estimate)	
Number of attempts before success	

Game 2: 12-Queens Challenge

Platform: [Online N-Queens Game](https://www.n-queens.com)

Direct Link (click to play): <https://www.n-queens.com>

AI Concept / Domain: Constraint Satisfaction Problem (CSP) — solved with backtracking search, optionally accelerated with constraint propagation and variable-ordering heuristics.

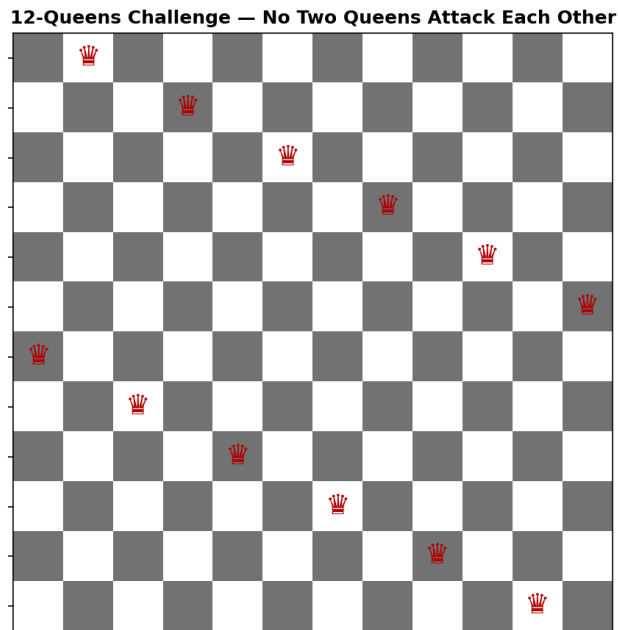


Fig. 4.1 — A 12×12 board with queens placed so no two share a row, column, or diagonal.

Scenario

You act as an AI chess strategist tasked with placing 12 queens on a 12×12 chessboard. A queen attacks any piece sharing its row, column, or diagonal. You must place all 12 queens so that no two attack each other — a classic combinatorial problem whose search space grows extremely fast as the board size increases (there are over 700 million ways to place 12 non-attacking queens' row-assignments to check naively, though constraint-aware search explores only a tiny fraction of these).

Objective of the Activity

Place all 12 queens on the board such that zero pairs of queens attack each other, ideally arriving at a valid configuration using as few trial placements or corrections as possible.

Initial State & Goal State

Initial State: an empty 12×12 board with no queens placed. Goal State: a board with exactly 12 queens placed, satisfying the constraint that no two share a row, column, or diagonal.

Actions / Operators Available

- Place a queen on an empty square
- Remove a queen from its current square

Algorithm Walkthrough — Backtracking Search

1. Work one row at a time; for Row 1, try placing a queen in Column 1.
2. Before placing a queen in any row, check that its column and both diagonals are free of existing queens.
3. If a safe column is found, place the queen and move to the next row.
4. If no column in the current row is safe, backtrack: remove the queen from the previous row and try its next untried column.
5. Repeat until all 12 rows have a safely placed queen (solution found) or every option has been exhausted (no solution from this branch).

Worked Example Trace

Row 1: place a queen in column 2 (safe, board otherwise empty).

Row 2: column 1 conflicts (diagonal), column 2 conflicts (same column) — column 4 is safe, place it there.

Row 3: columns 1, 2, 3, 5, 6 all conflict with the two placed queens — suppose every column conflicts; backtrack to Row 2 and try the next safe column instead of 4.

This place-check-backtrack cycle continues row by row until all 12 rows hold a mutually safe queen.

Row Being Filled	Column Tried	Conflict?	Action
1	2	No	Place queen, move to Row 2
2	1	Yes (diagonal)	Try next column
2	4	No	Place queen, move to Row 3
3	1,2,3,5,6...	Yes (all conflict)	Backtrack to Row 2
2	next available column	No	Place queen, continue
...	...	...	Continue until Row 12 is filled

Complexity & Performance Notes

Naive brute force would try every way of placing 12 queens on 144 squares — an astronomically large number. Restricting to one queen per row cuts this down to 12^{12} row/column combinations, and backtracking with per-placement conflict checks prunes this further by abandoning invalid partial boards immediately.

Approach	Time Complexity	Space Complexity	Optimal?
Brute force (all placements)	$O(n^{(2n)})$	$O(n)$	Yes, if it finishes

Approach	Time Complexity	Space Complexity	Optimal?
One-queen-per-row search	$O(n^n)$	$O(n)$	Yes
Backtracking with conflict check	Much less than n^n in practice	$O(n)$	Yes
Min-Conflicts local search	Often near-linear in practice	$O(n)$	Yes (usually, not guaranteed)

Alternative Approaches You Could Try

- Min-Conflicts Local Search — start with all queens placed randomly (one per row), then repeatedly move the most-conflicted queen to the column that minimises conflicts; this heuristic method solves even the 1,000,000-queens problem in seconds.
- Constraint Propagation (Forward Checking) — after placing a queen, immediately remove now-illegal columns from every future row's domain, catching dead ends earlier than plain backtracking.
- Genetic Algorithms — treat each board as a 'chromosome' and evolve a population of boards toward zero conflicts using selection, crossover, and mutation.

How Playing This Helps You Learn AI

- Introduces 'constraints' as rules that prune the search space during the search, not just checks applied at the very end.
- Shows why brute-force placement becomes impractical as board size grows, motivating smarter techniques like backtracking with early conflict detection.
- Builds pattern-recognition for row/column/diagonal relationships, mirroring how a CSP solver tracks conflicts internally.
- Reinforces that real scheduling and resource-allocation problems (timetabling, Sudoku, register allocation) are CSPs at their core.

Common Mistakes to Avoid

- Placing all queens first and only checking conflicts at the end — this wastes far more effort than checking as you go.
- Forgetting to check both diagonals (not just rows and columns).
- Not backtracking far enough — sometimes the conflict traces back more than one row.

Real-World Applications of This Technique

- Class-timetable and exam-scheduling systems that must avoid room/time clashes.
- Register allocation in compilers, where variables must not conflict for the same hardware register.
- Sudoku solvers and other logic-puzzle engines.

Frequently Asked Questions

Q: Why place only one queen per row?

A: Since no two queens can share a row anyway, restricting the search to exactly one queen per row eliminates a huge number of invalid configurations from ever being considered.

Q: Is there always a solution for any board size?

A: Yes, for all $n \geq 4$ (the N-Queens problem has no solution for $n = 2$ or $n = 3$).

Q: Why does backtracking sometimes feel slow near the end?

A: The last few rows have the fewest safe columns remaining, so conflicts are more likely to be discovered late, forcing backtracking through several rows at once.

Tips to Play & Solve Efficiently

- Place queens row by row; before committing, check that column and both diagonals are free.
- If you hit a dead end, backtrack to the previous row and try the next available column — this is backtracking in action.
- Look for a placement pattern (e.g., knight's-move-style offsets) that scales to any board size.

Reflection Questions for Your Report

1. How many times did you have to backtrack before finding a valid configuration?
2. Which constraint (row, column, or diagonal) caused you the most conflicts, and why?
3. How would you change your strategy if the board size were increased to 16×16 ?

Fillable Score Sheet — Record Your Own Attempt

Use this table while you play to capture the data you will need for the 'Game Performance' and 'Efficiency and Optimality' sections of your report.

Field	Your Recorded Value
Total placement attempts	
Number of backtracks	
Time taken (mm:ss)	
Final number of conflicts (should be 0)	
Number of full attempts before success	

5. Game 3: Water Jug Puzzle

Platform: [Tic-Tac-Toe \(hosting site\)](https://playtictactoe.org)

Direct Link (click to play): <https://playtictactoe.org>

AI Concept / Domain: State-Space Search — each (amount in Jug A, amount in Jug B) pair is a state; BFS guarantees the fewest actions to the goal.

Water Jug Puzzle — Target: Exactly 8 Litres

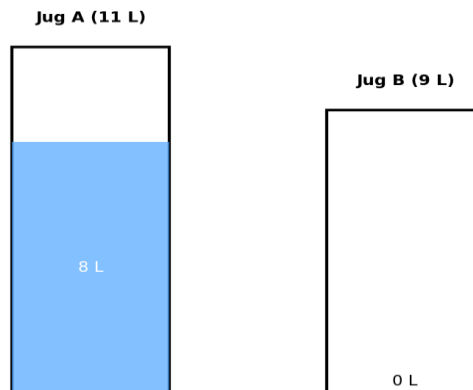


Fig. 5.1 — Jug A (11 L) and Jug B (9 L); the goal is to isolate exactly 8 litres in either jug.

Scenario

You are given two unmarked jugs — one holding up to 11 litres, the other up to 9 litres. Using only three permitted actions (fill a jug completely, empty a jug completely, or pour from one jug into the other until either the source is empty or the destination is full), you must find a sequence of actions that leaves exactly 8 litres in one of the jugs. Every combination of water levels is a distinct 'state', and moving between states by filling, emptying, or pouring is exactly how state-space search problems are framed.

Objective of the Activity

Starting from both jugs empty, perform a sequence of fill/empty/pour actions that results in exactly 8 litres of water in either Jug A or Jug B, using the fewest actions possible (Level 19 of the puzzle).

Initial State & Goal State

Initial State: (0, 0) — both jugs empty. Goal State: any state (8, x) or (x, 8) for any valid x.

Actions / Operators Available

- Fill Jug A completely (to 11 L)
- Fill Jug B completely (to 9 L)
- Empty Jug A completely
- Empty Jug B completely
- Pour Jug A into Jug B (until A is empty or B is full)
- Pour Jug B into Jug A (until B is empty or A is full)

Algorithm Walkthrough — Breadth-First Search over (A, B) states

1. Represent the current situation as a pair (amount in A, amount in B).
2. From the current state, generate every state reachable by one of the six actions.
3. Add any newly-seen state to a queue, and remember which action produced it.
4. Process the queue in first-in-first-out order (this guarantees the fewest actions to any given state).
5. Stop as soon as a state with $A = 8$ or $B = 8$ is generated; trace back the recorded actions to get the solution sequence.

Worked Example Trace

$(0,0) \rightarrow \text{Fill A} \rightarrow (11,0) \rightarrow \text{Pour A into B} \rightarrow (2,9) \rightarrow \text{Empty B} \rightarrow (2,0) \rightarrow \text{Pour A into B} \rightarrow (0,2) \rightarrow \text{Fill A} \rightarrow (11,2) \rightarrow \text{Pour A into B} \rightarrow (4,9) \rightarrow \text{Empty B} \rightarrow (4,0) \rightarrow \text{Pour A into B} \rightarrow (0,4) \rightarrow \text{Fill A} \rightarrow (11,4) \rightarrow \text{Pour A into B} \rightarrow (6,9) \rightarrow \text{Empty B} \rightarrow (6,0) \rightarrow \text{Pour A into B} \rightarrow (0,6) \rightarrow \text{Fill A} \rightarrow (11,6) \rightarrow \text{Pour A into B} \rightarrow (8,9) \dots$ at $(8,9)$ Jug A already reads 8 — goal reached.

Note there are usually several valid sequences of different lengths; the assessment asks you to find one that uses the minimum number of actions, which is exactly what BFS guarantees by exploring shorter sequences before longer ones.

Step	Action	Jug A (11L)	Jug B (9L)
0	Start	0	0
1	Fill A	11	0
2	Pour A $\rightarrow$ B	2	9
3	Empty B	2	0
4	Pour A $\rightarrow$ B	0	2
...	... repeat pattern ...	...	...
n	Pour A $\rightarrow$ B	8	9

Complexity & Performance Notes

Because each jug can hold between 0 and its capacity, the total number of distinct states is bounded by $(\text{capacity of A} + 1) \times (\text{capacity of B} + 1) = 12 \times 10 = 120$ possible states — small enough that BFS explores the full space almost instantly.

Approach	Time Complexity	Space Complexity	Optimal?
Brute force / random tries	Unbounded	$O(1)$	No
DFS over states	$O(\text{states})$ worst case	$O(\text{depth})$	No — may overshoot with a longer sequence
BFS over states	$O(\text{states}) = O(120)$	$O(\text{states})$	Yes — fewest actions

Alternative Approaches You Could Try

- DFS — will eventually find a valid sequence but is not guaranteed to be the shortest one.
- Bidirectional Search — search forward from (0,0) and backward from any goal state (8,x)/(x,8) simultaneously, meeting in the middle.
- Mathematical (Number-Theoretic) Approach — using the identity that reachable totals are exactly the multiples of $\gcd(11,9) = 1$ up to 11, confirming any integer target from 0–11 is theoretically reachable, though the actual action sequence still needs to be found by search.

How Playing This Helps You Learn AI

- Turns an abstract search-tree diagram into a tangible, hands-on puzzle — each action visibly changes the 'state'.
- Demonstrates the difference between a huge theoretical action space and the much smaller set of states actually useful toward the goal.
- Encourages systematic exploration (try an action, note the new state, don't repeat visited states).
- Helps you appreciate why an algorithm tracks 'visited states' to avoid wasted, repeated work.

Common Mistakes to Avoid

- Repeating a state you've already reached, which wastes moves without making progress.
- Pouring without checking the destination jug's remaining capacity, leading to an unintended state.
- Giving up on a branch too early instead of systematically trying all six actions from each state.

Real-World Applications of This Technique

- Industrial process control problems involving mixing and measuring fixed-capacity tanks.
- Puzzle-solving and planning modules in AI teaching tools and automated theorem provers.
- General resource-transfer optimisation problems with discrete capacities.

Frequently Asked Questions

Q: Why must I track visited states?

A: Without tracking, you risk repeating a state you've already tried (e.g., emptying a jug you just emptied), which wastes moves without making progress toward the goal.

Q: Does the order of pouring matter?

A: Yes — pouring A into B versus B into A from the same state produces different resulting states, so both must be considered as separate possible actions.

Q: What is the minimum number of actions for this specific puzzle?

A: It depends on the exact capacities and target, but BFS over the state graph will always report the true minimum for you.

Tips to Play & Solve Efficiently

- Write down every state you reach as an ordered pair (A, B) so you don't repeat one already tried.
- Try working backward from the goal state (8, x) to see which action would have produced it.
- The greatest common divisor of the two jug capacities tells you which target amounts are even reachable.

Reflection Questions for Your Report

1. How many actions did your solution use, and can you find a shorter sequence?
2. Which states did you visit more than once by accident, and how would tracking 'visited states' have prevented that?
3. Is a target of 10 litres reachable with an 11 L and a 9 L jug? Justify your answer using the GCD rule.

Fillable Score Sheet — Record Your Own Attempt

Use this table while you play to capture the data you will need for the 'Game Performance' and 'Efficiency and Optimality' sections of your report.

Field	Your Recorded Value
Total actions taken	
Time taken (mm:ss)	
Number of repeated states encountered	
Minimum actions your BFS trace found (for comparison)	
Number of attempts before success	

Game 4: Connect Four AI Challenge

Platform: [Connect Four \(MathsIsFun\)](https://www.mathsisfun.com/games/connect4.html)

Direct Link (click to play): <https://www.mathsisfun.com/games/connect4.html>

AI Concept / Domain: Adversarial Search — Minimax, typically enhanced with Alpha-Beta Pruning.

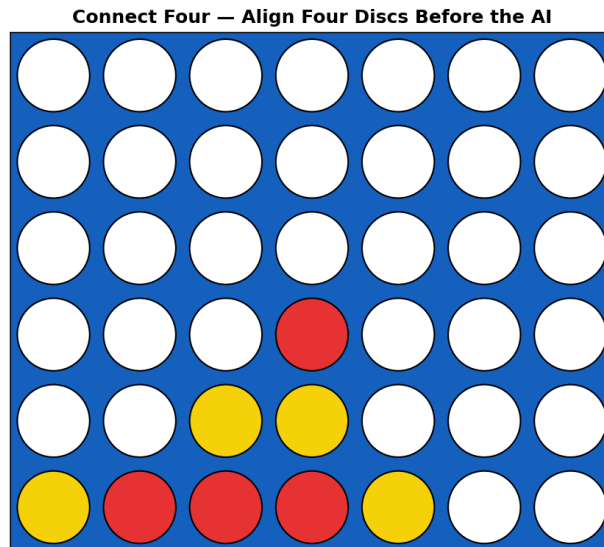


Fig. 6.1 — Two players alternate dropping discs; the first to align four in a row wins.

Scenario

You play against a computer-controlled opponent on a 7-column, 6-row vertical grid. Players alternate dropping a disc into a chosen column; gravity pulls it to the lowest empty row in that column. The first player to align four discs — horizontally, vertically, or diagonally — wins. Because the opponent is actively working against your goal, this differs fundamentally from the earlier puzzles: you must anticipate and counter another decision-maker's moves, which is the essence of adversarial (game-tree) search.

Objective of the Activity

Connect four of your own discs in a row before the AI opponent connects four of its own, while proactively blocking the AI's near-complete lines.

Initial State & Goal State

Initial State: an empty 7×6 grid. Goal State: a grid position containing four of your discs in an unbroken horizontal, vertical, or diagonal line.

Actions / Operators Available

- Drop a disc into any column that is not already full

Algorithm Walkthrough — Minimax with Alpha-Beta Pruning

1. From the current board, generate every possible move (one per non-full column).
2. For each resulting board, recursively imagine the opponent's best reply, then your best reply to that, down to a fixed search depth or until the game ends.
3. Assign a numeric score to each end position (e.g., +1000 if you win, -1000 if the AI wins, 0 for a draw, or a heuristic estimate at the search depth limit).
4. Propagate scores back up the tree: at MIN levels (opponent's turn) take the minimum child score; at MAX levels (your turn) take the maximum child score.
5. Choose the move at the root that leads to the highest propagated score.
6. (Optimisation) Use alpha-beta pruning to stop evaluating a branch as soon as it is proven irrelevant to the final decision, without changing the chosen move.

Worked Example Trace

Suppose you can drop a disc in column 3 or column 5. Simulating column 3 shows the AI's best reply still leaves you able to force a win two moves later (score +1). Simulating column 5 shows the AI can force a draw at best (score 0). Minimax therefore selects column 3, because MAX prefers the branch with the higher guaranteed score.

If, while exploring column 5's subtree, you discover a branch that is already worse than the +1 you found under column 3, alpha-beta pruning lets you stop exploring that branch immediately — the final decision (column 3) does not change, but far fewer positions had to be evaluated.

Ply	Player	Move Considered	Propagated Value
1 (root)	MAX (You)	Column 3	score = max(children)
2	MIN (AI reply)	Column 5 (blocking)	score = min(children)
3	MAX (Your reply)	Column 3 again	leaf evaluation
leaf	—	Terminal / depth limit reached	+1 / 0 / -1000 etc.

Complexity & Performance Notes

Connect Four has at most 7 choices per turn and a maximum of 42 discs, giving an enormous full game tree. Minimax with a fixed search depth d explores roughly 7^d positions; alpha-beta pruning with good move-ordering can explore closer to $7^{(d/2)}$ positions for the same result.

Approach	Time Complexity	Space Complexity	Optimal?
Full game tree (solved)	$O(7^{42})$ theoretical	$O(42)$	Yes (proven first-player win)
Minimax to depth d	$O(7^d)$	$O(d)$	Optimal within depth d

Approach	Time Complexity	Space Complexity	Optimal?
Minimax + Alpha-Beta	$O(7^{(d/2)})$ with good ordering	$O(d)$	Same result as plain minimax, faster

Alternative Approaches You Could Try

- Monte Carlo Tree Search (MCTS) — instead of exhaustively evaluating a tree, randomly simulate many playouts from each candidate move and pick the move with the best average outcome; used in modern game-playing systems like AlphaGo.
- Iterative Deepening Minimax — run minimax to depth 1, then 2, then 3, and so on, stopping when time runs out, so you always have a 'best move so far' available.
- Heuristic Evaluation Functions — instead of only scoring wins/losses/draws, score partial positions (e.g., +1 per open three-in-a-row) so minimax can make good decisions without searching to the end of the game.

How Playing This Helps You Learn AI

- Introduces the minimax idea directly: for every move you consider, you should also consider the opponent's best possible reply.
- Builds intuition for 'look-ahead depth' — stronger play comes from thinking several moves ahead rather than reacting only to the current board.
- Demonstrates the value of alpha-beta pruning conceptually: once a move is clearly worse, stop analysing it and move to a better option.
- Highlights the offense/defense trade-off central to adversarial search — a move that helps you can simultaneously need to block your opponent.

Common Mistakes to Avoid

- Focusing only on your own potential wins while ignoring the AI's near-complete lines.
- Playing reactively (only responding to the last move) instead of planning two or three moves ahead.
- Filling the centre columns too late — they participate in the most possible winning lines.

Real-World Applications of This Technique

- Game-playing AI in chess, checkers, and other two-player games (historically, Deep Blue's chess engine used minimax with pruning).
- Automated negotiation and auction-bidding systems that must anticipate a competitor's response.
- Security and military strategic-planning simulations that model an adversary's best response.

Frequently Asked Questions

Q: Why does the AI sometimes seem to 'give up' a winning line?

A: It may be sacrificing a slower win for a faster or safer one, or may have a limited search depth and not see the far consequence.

Q: Is Connect Four a solved game?

A: Yes — it has been mathematically proven that the first player can force a win with perfect play, particularly by starting in the centre column.

Q: Does alpha-beta pruning change what move is chosen?

A: No — it always returns the same decision as plain minimax; it only skips work that could not have changed the outcome.

Tips to Play & Solve Efficiently

- Control the centre column early — it participates in the most possible four-in-a-row lines.
- Always check whether the AI has three in a row with an open end before making your own move.
- Try to create a 'double threat' — two different ways to win at once, which the opponent cannot block simultaneously.

Reflection Questions for Your Report

1. Describe one moment where you had to choose between attacking and blocking. What did you choose, and why?
2. How many moves ahead do you think you were 'looking' when you played your best move of the game?
3. How would alpha-beta pruning change the number of positions the AI needs to evaluate, without changing its final decision?

Fillable Score Sheet — Record Your Own Attempt

Use this table while you play to capture the data you will need for the 'Game Performance' and 'Efficiency and Optimality' sections of your report.

Field	Your Recorded Value
Result (Win / Loss / Draw)	
Total moves in the game	
Time taken (mm:ss)	
Number of blocking moves you made	
Number of games played before a win	

Game 5: 8-Puzzle AI Challenge

Platform: [8-Puzzle Game](https://8puzzle.org/)

Direct Link (click to play): <https://8puzzle.org/>

AI Concept / Domain: Heuristic (Informed) Search — A* using Manhattan Distance or Misplaced-Tiles heuristics.

Initial (Scrambled) State			Goal State		
7	2	4	1	2	3
5		6	4	5	6
8	3	1	7	8	

Fig. 7.1 — A scrambled 3×3 tile arrangement (left) must be rearranged into the ordered goal state (right).

Scenario

You are given a 3×3 grid containing eight numbered tiles and one blank space, scrambled into a random arrangement. By repeatedly sliding a tile adjacent to the blank into that blank, you must rearrange the tiles into an ordered sequence (1 through 8, with the blank last). Because only tiles next to the blank can move, this is a constrained search problem where the number of possible arrangements (181,440 reachable states) is large enough that informed search is far more efficient than blind trial-and-error.

Objective of the Activity

Transform the scrambled tile arrangement into the ordered goal arrangement using the minimum possible number of tile slides.

Initial State & Goal State

Initial State: the given scrambled 3×3 arrangement of tiles 1–8 and one blank. Goal State: tiles in numerical order with the blank in the bottom-right position.

Actions / Operators Available

- Slide the tile above the blank downward
- Slide the tile below the blank upward
- Slide the tile left of the blank rightward
- Slide the tile right of the blank leftward

Algorithm Walkthrough — A* Search with Manhattan Distance Heuristic

1. For the current arrangement, compute $g(n)$: the number of slides made so far.
2. Compute $h(n)$: the sum, over every tile, of how many rows plus columns it is away from its goal position (Manhattan distance).
3. Compute $f(n) = g(n) + h(n)$ for every arrangement reachable by one slide from the current one.
4. Always expand next the arrangement in your frontier with the lowest $f(n)$.
5. Repeat until the arrangement with $h(n) = 0$ (the goal) is expanded; trace back the moves that produced it.

Worked Example Trace

Take the scrambled state 7-2-4 / 5-_-6 / 8-3-1 shown in Fig. 7.1. Tile 1 is at the bottom-right area but belongs in the middle row; tile 7 is top-left but belongs bottom-left — summing every tile's distance from its goal position gives the heuristic estimate $h(n)$ for this state.

A* first tries the slide that most reduces this total heuristic distance (for instance, moving tile 6 or tile 5 toward its home row), then re-evaluates $h(n)$ for the new arrangement, and continues greedily-but-optimally toward the lowest combined $f(n)$ until the ordered goal state is reached.

Step	Move	$g(n)$	$h(n)$ (Manhattan)	$f(n) = g+h$
0	Start	0	12	12
1	Slide 6 up	1	10	11
2	Slide 5 left	2	9	11
3	Slide 1 up	3	7	10
...	...	...	...	...
k	Final slide	k	0	k

Complexity & Performance Notes

The 8-Puzzle has $9!/2 = 181,440$ reachable states. Blind search (BFS) must potentially explore a large fraction of these, while A* with an admissible heuristic like Manhattan Distance typically expands only a small, focused subset on its way to the goal.

Approach	Time Complexity	Space Complexity	Optimal?
Brute force / random moves	Unbounded	$O(1)$	No
BFS (blind)	$O(181,440)$ worst case	$O(181,440)$	Yes, but slow
A* + Misplaced Tiles heuristic	Much less than BFS in practice	$O(\text{open} + \text{closed lists})$	Yes (heuristic is admissible)

Approach	Time Complexity	Space Complexity	Optimal?
A* + Manhattan Distance heuristic	Fewer expansions than Misplaced Tiles	O(open+closed lists)	Yes (stronger admissible heuristic)

Alternative Approaches You Could Try

- IDA* (Iterative Deepening A*) — repeats depth-limited DFS with an increasing $f(n)$ cutoff, achieving A*-quality solutions using far less memory than storing a full open list.
- Greedy Best-First Search — expand purely by $h(n)$, ignoring $g(n)$; usually faster but not guaranteed optimal, since it can be misled by the heuristic.
- Bidirectional A* — search simultaneously from the scrambled state and from the goal state, meeting in the middle to cut the effective search depth roughly in half.

How Playing This Helps You Learn AI

- Makes the value of a heuristic tangible: a quick 'how close am I?' estimate is exactly what A* uses to prioritise which state to expand next.
- Shows why blind search becomes impractical here, while heuristic-guided search reaches the goal in far fewer steps.
- Builds spatial planning skill — deciding which tile to move now to avoid undoing progress later.
- Provides a natural comparison: your number of moves versus the A*-optimal number of moves for the same scramble.

Common Mistakes to Avoid

- Moving the same tile back and forth, effectively undoing your own progress.
- Focusing on getting one tile into place while accidentally displacing tiles that were already correct.
- Ignoring the bottom row/right column until the very end, which often forces extra corrective moves.

Real-World Applications of This Technique

- Route-finding in GPS systems where the heuristic is straight-line distance to the destination.
- Robotics motion planning, where A* finds efficient paths through obstacle-filled spaces.
- Puzzle and logistics solvers (e.g., warehouse robot tile/box rearrangement).

Frequently Asked Questions

Q: Why is Manhattan Distance considered 'better' than Misplaced Tiles?

A: Manhattan Distance gives a larger (tighter) but still admissible estimate of remaining cost, so A* needs to expand fewer states to reach the same optimal solution.

Q: Can every scrambled 8-Puzzle be solved?

A: No — exactly half of all possible tile arrangements are solvable from any given goal state, depending on the parity of the permutation.

Q: What does 'admissible' mean in practice here?

A: It means the heuristic's estimate is never larger than the actual number of moves still required, which is what guarantees A* won't miss a better (shorter) solution.

Tips to Play & Solve Efficiently

- Solve the top row first, then the left column, before untangling the remaining 2×2 block.
- Before each move, estimate the Manhattan distance of the resulting state; prefer moves that reduce this total.
- Avoid moving the same tile back and forth — track your last two moves to prevent immediate undo loops.

Reflection Questions for Your Report

1. How many moves did you use, and how does that compare with the Manhattan-distance estimate for the starting scramble?
2. Which heuristic — misplaced tiles or Manhattan distance — do you think gives a more accurate estimate, and why?
3. Describe a moment where a move that looked good short-term made a later step harder.

Fillable Score Sheet — Record Your Own Attempt

Use this table while you play to capture the data you will need for the 'Game Performance' and 'Efficiency and Optimality' sections of your report.

Field	Your Recorded Value
Total slides used	
Time taken (mm:ss)	
Manhattan-distance estimate at start	
Difference between your moves and the estimate	
Number of attempts before success	

Cross-Game Comparison Table

The table below summarises how the five activities map onto the major families of AI search covered by CO1.

Game	AI Concept	Algorithm	Guarantees Optimal?	Key Data Structure
AI Maze Escape	State-Space Search	BFS / DFS	BFS: Yes DFS: No	Queue (BFS) / Stack (DFS)
12-Queens Challenge	Constraint Satisfaction	Backtracking	Yes (finds valid solution)	Partial assignment + conflict check
Water Jug Puzzle	State-Space Search	BFS	Yes (fewest actions)	Queue of (A,B) states
Connect Four	Adversarial Search	Minimax + Alpha-Beta	Optimal within search depth	Game tree
8-Puzzle	Heuristic Search	A* (Manhattan Distance)	Yes, if heuristic admissible	Priority queue (open list)

Notice the progression across the five games: from no extra information at all (Maze, blind BFS/DFS), to structural rules that prune the search (Queens, CSP constraints), to a small but exhaustively searchable state space (Water Jug), to an opponent actively working against you (Connect Four), and finally to a numeric estimate guiding the search (8-Puzzle, heuristic function). Together they form a compact tour of the AI search techniques covered in CO1.

Glossary of Key AI Terms

Term	Definition
State	A complete description of the problem at one point in time.
State Space	The set of all states reachable from the initial state via legal actions.
Operator / Action	A rule that transforms one state into another.
Initial State	The state the problem begins in.
Goal State	A state (or set of states) that satisfies the problem's success condition.
Frontier / Open List	The set of states that have been generated but not yet expanded.
Visited / Closed List	The set of states that have already been expanded, to avoid repeating work.
Branching Factor	The average number of successor states generated from each state.
Heuristic Function $h(n)$	An estimate of the cost from state n to the nearest goal state.
Admissible Heuristic	A heuristic that never overestimates the true remaining cost, guaranteeing A* finds an optimal solution.
Constraint Satisfaction Problem (CSP)	A problem defined by variables, domains, and constraints restricting allowed value combinations.
Backtracking	A depth-first strategy that abandons a partial solution as soon as it violates a constraint.
Minimax	An adversarial-search algorithm that alternates maximising and minimising outcome values at each level of a game tree.
Alpha-Beta Pruning	An optimisation of minimax that skips branches proven irrelevant to the final decision.
Optimality	A property of an algorithm that guarantees the solution found has the lowest possible cost.
Completeness	A property of an algorithm that guarantees it will find a solution if one exists.
Path Cost	The total number of actions (or their weighted sum) taken to travel from the initial state to a given state.
Frontier Expansion	The process of generating all successor states of the state currently being explored.
Game Tree	A tree representation of a game in which nodes are board positions and edges are legal moves.
Ply	A single move by one player in a game tree (two plies make one full 'turn' in a two-player game).

Term	Definition
Evaluation Function	A heuristic function that estimates how good a game position is for a player, used when the search cannot reach a terminal state.
Local Search	A search strategy (e.g., Min-Conflicts, Hill-Climbing) that moves between complete candidate solutions rather than building a solution incrementally.
Manhattan Distance	The sum of the absolute horizontal and vertical distances between a tile's current position and its goal position.
Time Complexity	A measure of how the running time of an algorithm grows as the size of its input (or state space) grows.

How to Write Up Each Section of Your Report

The official report template (Annexure A) lists 17 components. Here is how the material in this guide maps onto each one:

Report Component	Where This Guide Helps
Title of the Game-Based Activity	Use the game's name and the specific level/challenge you attempted.
Game Platform / Link	Copy the Platform and Direct Link given at the start of each chapter.
Objective of the Activity	Adapt the 'Objective of the Activity' paragraph to your specific attempt.
Problem Statement	Summarise the 'Scenario' section in your own words.
AI Concept / Domain	State the AI Concept named at the top of the chapter (e.g., 'Constraint Satisfaction Problem').
Initial State and Goal State	Copy and, if needed, adapt the formal Initial/Goal State description to your specific play-through.
Actions / Operators Used	List the operators given in the chapter; note which ones you actually used.
Solution Strategy	Summarise the Algorithm Walkthrough in your own words, describing how you (or the ideal algorithm) approached the problem.
Step-by-Step Solution	Record your own actual sequence of moves — use the Worked Example Trace as a model for the level of detail expected.
Game Performance	Note your number of attempts, moves, and time — this is personal to your play-through.
Successful Completion and Evidence	Attach your own screenshots; this cannot be provided in a study guide.
Efficiency and Optimality	Compare your move count against the optimal count discussed in the chapter.
Analysis and Interpretation	Draw on the 'How Playing This Helps You Learn AI' section, applied to your specific experience.
Critical Thinking and Alternative Solution	Answer the 'Reflection Questions' — these are designed to prompt exactly this kind of discussion.
Learning Outcome / Reflection	Reflect personally on which Reflection Question resonated most and why.
Conclusion	Summarise your outcome and connect it back to the AI Concept named for that chapter.
References	Cite the platform link plus any of the further reading in Chapter 11.

Appendix A: Official Evaluation Rubric (Reproduced from Assessment Tool 3)

For quick reference while writing your report, the full four-band evaluation rubric from the official assessment document is reproduced below.

Criteria (%)	Excellent	Good	Satisfactory	Needs Improvement
Understanding of Problem/Objective (10%)	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search Technique (15%)	Correctly applies the appropriate technique	Applies the technique with minor errors	Shows limited application	Unable to apply the appropriate technique
Successful Completion (20%)	Solves/completes independently	Completes with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency / Optimal Moves (10%)	Optimal or highly efficient moves	Reasonably efficient sequence	Several unnecessary moves	Highly inefficient or incomplete
Documentation / Evidence (15%)	Clear steps, score, screenshots, evidence	Provides most required evidence	Incomplete steps or evidence	No proper documentation
Analysis & Interpretation (15%)	Clearly analyses and explains technique/outcome	Good analysis with minor gaps	Limited analysis or explanation	Unable to analyse or interpret
Originality & Critical Thinking (15%)	Excellent independent thinking, explores alternatives	Good independent thinking	Limited critical thinking	Little or no independent thinking

Appendix B: Rubric Self-Check Before Submission

Before submitting your report, use this checklist against the official Evaluation Rubrics to confirm each criterion is addressed for every one of the five games.

Rubric Criterion	Weight	Have I addressed this for all 5 games?
Understanding of the Problem / Game Objective	10%	☐ Yes ☐ No
Application of AI Search / Problem-Solving Technique	15%	☐ Yes ☐ No
Successful Completion of the Game / Puzzle	20%	☐ Yes ☐ No
Efficiency of Solution / Optimal Moves	10%	☐ Yes ☐ No
Documentation / Evidence of Completion	15%	☐ Yes ☐ No
Analysis and Interpretation of the Solution	15%	☐ Yes ☐ No
Originality and Critical Thinking	15%	☐ Yes ☐ No

A quick per-game reminder of what 'Excellent' looks like on the two criteria students most often under-develop:

Game	What 'Excellent' Analysis Looks Like	What 'Excellent' Critical Thinking Looks Like
Maze Escape	Explicitly names BFS/DFS and explains why your path was or wasn't shortest.	Proposes A* or bidirectional search as a faster alternative and explains why.
12-Queens	Explains backtracking and identifies exactly which constraint caused each conflict.	Discusses Min-Conflicts or forward checking as a faster alternative for larger boards.
Water Jug	Shows the explicit state sequence (A,B) and confirms it against the GCD reachability rule.	Proposes a shorter sequence, or explains why the one found is already minimal via BFS.
Connect Four	Identifies at least one moment of look-ahead reasoning and one blocking decision.	Explains how alpha-beta pruning or a heuristic evaluation function would change the AI's efficiency.
8-Puzzle	States the heuristic used and compares your move count to the heuristic's initial estimate.	Compares Misplaced Tiles vs Manhattan Distance, or proposes IDA* for memory efficiency.

Appendix C: Code of Conduct & Marks Sheet (Reproduced for Convenience)

This reproduces the declaration and marks-recording layout from the official Assessment Tool 3 document, so you can review the exact wording before signing your real submission.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Analysis and Interpretation of the Solution	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty: _____

Total Marks Awarded:

References & Further Reading

Game platforms used in this assessment:

- [AI Maze Escape — Scratch](#)
- [12-Queens Challenge — Online N-Queens Game](#)
- [Water Jug Puzzle — Tic-Tac-Toe hosting site](#)
- [Connect Four AI Challenge — MathsIsFun](#)
- [8-Puzzle AI Challenge](#)

Suggested background reading on the underlying algorithms (classical AI textbooks and reference material — consult your course library or instructor for full citations):

- Russell, S. & Norvig, P. — Artificial Intelligence: A Modern Approach (chapters on Search, CSPs, and Adversarial Search).
- Course lecture notes for MLA01 — Artificial Intelligence and Expert Systems, State-Space Search and Heuristic Techniques (CO1).
- Standard references on the A* algorithm, Minimax with Alpha-Beta Pruning, and Backtracking Search from any introductory AI course.